

Patrick Kramer
CS594
Portland State University
May 2026

Internet Relay Chat Project

Status of this Memo

This memo describes the class project I did for my Internetworking Protocols class in the Spring term of 2026 at Portland State University. This is not an internet standard, simply a student's recreation of used networking systems to better understand concepts learned in class.

Abstract

This document specifies a simplified version of an Internet Relay Chat (IRC) protocol designed for a client-server chat application. It defines the commands and their formats the client can issue and how the server behaves to support functionalities such as room creation, room membership, and messaging, among others. This is intended for educational use and does not include common modern features such as user authentication, data encryption, or many other advanced features modern chat applications have.

Introduction

Internet Relay Chat (IRC) is a teleconferencing system that has existed for several decades and enables lightweight, real-time, text-based communication. This document describes a simplified IRC-style protocol implemented as part of an academic project. The system allows multiple clients to communicate through an intermediary server that receives, interprets, and distributes all client-issued commands. Messages exchanged through this protocol are plain text and transmitted over TCP/IP, leveraging TCP's reliability and ordered delivery to ensure correct message transmission without requiring high-performance networking.

This document defines the commands, message formats, and server behaviors required to support the core functionality of the chat system. Supported operations include client-server connection establishment, room creation, room listing, joining and leaving rooms, listing room members, sending messages to one or more rooms, and handling client and server disconnections.

System Architecture

The chat system described in this document follows a client–server architecture in which all communication between clients is mediated by a central server. Clients do not communicate directly with one another; instead, each client establishes a TCP connection to the server, sends text-based commands, and receives responses or forwarded messages from the server.

The server maintains all global state for the system, including the list of active rooms, the membership of each room, and the set of currently connected clients. When a client issues a command—such as creating a room, joining a room, or sending a message—the server interprets the command, updates its internal state as needed, and distributes messages to the appropriate recipients.

The server is responsible for:

- Accepting and managing multiple simultaneous client connections
- Parsing and validating client commands
- Creating, listing, and deleting rooms
- Tracking room membership
- Broadcasting messages to all clients within a room
- Handling client disconnections and cleaning up associated state

Clients are responsible for:

- Establishing a TCP connection to the server
- Sending properly formatted commands
- Receiving and displaying server responses and broadcast messages
- Handling server-initiated disconnections

This architecture ensures that all clients share a consistent view of the system and that message delivery is coordinated through a single authoritative server.

Client Message Format

All communication between clients and the server use a simple text-based format transmitted over a persistent TCP connection once the client has connected until they disconnect. Each message is a list of strings separated by whitespace, with the first string being a command and each subsequent string being arguments for that command. Messages are ended with a carriage-return/new-line sequence.

General Structure

Communication Template: <COMMAND> <Arg1> <Arg2> ... <ArgN>\r\n

- <COMMAND> is an uppercase keyword identifying the operation to perform.
- Arguments are positional and space-delimited.
- Commands that include free-form text (e.g., chat messages) treat the remainder of the line after the room name as the message body.
- Commands are case-insensitive, but the server will transmit responses using uppercase command names for consistency.

Whitespace Rules

- Commands and arguments are separated by a single ASCII space (0x20).
- Leading and trailing whitespace is not permitted.
- Multi-word message bodies are allowed only in commands that explicitly support them (e.g., SEND).
- The server does not preserve multiple consecutive spaces within message bodies.

Encoding

All messages use UTF-8 encoding. Characters outside the standard ASCII list may not be interpreted or rendered correctly by all clients.

Examples

- CREATE dnd_group\r\n
- JOIN dnd_group\r\n
- SEND dnd_group Hey! What snacks should I bring?\r\n
- LIST\r\n
- QUIT\r\n

Command Definitions

This section defines all commands that clients may issue to the server, along with the expected server behavior and example usage. All commands follow the message format described in Section 3.

For each command, the following information is provided:

- **Syntax** — the required structure of the command
- **Parameters** — arguments and their meanings
- **Description** — purpose of the command
- **Server Behavior** — how the server processes the command
- **Examples** — valid client messages

CREATE

Syntax:

```
CREATE <room_name>\r\n
```

Parameters:

<room_name> - the name of the room to create

Description:

Creates a new chatroom. If a room with the given name already exists the server will return an error.

Server Behavior:

If a room with the name <room_name> does not exist, create it and return a success response.

If a room with name <room_name> already exists, return ERR_ROOM_EXISTS.

The client is not automatically joined to the room.

Example:

```
CREATE foodies\r\n
```

LIST

Syntax:

```
LIST\r\n
```

Parameters:

None

Description:

Requests a list of all existing rooms on the server.

Server Behavior:

Returns a list of room names separated by newlines. If no rooms exist the list is empty.

Example:

LIST\r\n

JOIN

Syntax:

JOIN <room_name>\r\n

Parameters:

<room_name> - the name of the room to join

Description:

Adds the client to the room with the designated room_name. If the room does not exist, return an error.

Server Behavior:

If a room with the name <room_name> exists, add the client to the room.

If it does not exist, return ERR_NO_SUCH_ROOM

Example:

JOIN foodies\r\n

LEAVE

Syntax:

LEAVE <room_name>\r\n

Parameters:

<room_name> - the name of the room to leave

Description:

Removes the client from the room with the designated room_name.

Server Behavior:

If the client is in a room with the name <room_name>, remove the client from that room.

If the client is not in the room or the room does not exist, return ERR_NOT_IN_ROOM or ERR_NO_SUCH_ROOM respectively.

Notify the remaining members of the room that the client has left the room.

Example:

LEAVE foodies\r\n

MEMBERS

Syntax:

MEMBERS <room_name>\r\n

Parameters:

<room_name> - the name of the room to list members of

Description:

Returns a list of all clients currently in <room_name>.

Server Behavior:

If a room with name <room_name> exists, return a list of all current members.

If no room with name <room_name> exists, return ERR_NO_SUCH_ROOM.

Example:

MEMBERS foodies\r\n

SEND

Syntax:

SEND <room_name> <message>\r\n

Parameters:

<room_name> - the name of the room to send a message to

<message> - the message to send to members of the room

Description:

Broadcasts a message to all members of a specified room that the client is also a member of.

Server Behavior:

If the client is a member of <room_name>, broadcast <message> to all members.

If a room with name <room_name> does not exist, return ERR_NO_SUCH_ROOM.

If the client is not a member of <room_name> but it does exist, return ERR_NOT_IN_ROOM.

Example:

SEND foodies What's for dinner?\r\n

QUIT

Syntax:

QUIT\r\n

Parameters:

none

Description:

Disconnects the client from the server.

Server Behavior:

Remove the client from all rooms it is a member of.

Notify the members of each room that the client has left.

Close the TCP connection with the client.

Example:

QUIT\r\n

Server Responses

The server communicates with clients using the same text-based message format that clients use to communicate with the server. Server responses fall into three categories:

1. Success Responses - acknowledging to a client that their command was correctly processed.
2. Broadcast Messages - messages are sent to members of a room that another client in that room prompted.
3. Error Responses - notifications to the client that their command was not properly processed for some reason or another.

Success Responses

For commands from the client that are successfully processed, the server returns a confirmation message varying by the type of client command.

Syntax:

OK <command> <argument>\r\n

Examples:

OK ROOM_CREATED foodies\r\n

OK ROOM_JOINED foodies\r\n

OK MESSAGE_SENT foodies\r\n

OK LEFT_ROOM foodies\r\n

Broadcast Messages

When a client uses the SEND command, the server broadcasts the included message to all members of that room, including the sender. The message is preceded by a hyphen to help with readability separating the message from the message's metadata. The server will also broadcast messages to members of a group when another client joins or leaves.

Syntax:

```
MSG <room_name> <sender_id> <message>\r\n
NOTICE <room_name> <sender_id> JOINED\r\n
NOTICE <room_name> <sender_id> LEFT\r\n
```

Example:

```
NOTICE foodies client0 JOINED\r\n
MSG foodies client0 - What's for dinner?\r\n
NOTICE foodies client0 LEFT\r\n
```

Error Responses

When a client's command can't be processed, an error message is sent to the client with some sort of description regarding what went wrong.

Syntax:

```
ERR <error_code> <details>\r\n
```

Defined Error Codes:

ERR_UNKNOWN_COMMAND - the command is not recognized

ERR_INVALID_ARGS - The command's arguments were missing

ERR_NO_SUCH_ROOM - The specified does not exist

ERR_ROOM_EXISTS - Attempted to create a room that already exists

ERR_NOT_IN_ROOM - The client attempted a command requiring room membership while not a member.

ERR_NOT_CONNECTED - Client is not properly connected to the server

ERR_INTERNAL - A server-side error has occurred

Error Responses

When a client disconnects, the server sends notifications to all rooms the client was a member of. The server will also send a notification when shutting down.

Syntax:

```
NOTICE <room_name> <client> DISCONNECTED\r\n
NOTICE SERVER SHUTDOWN\r\n
```

Connection Handling

All communication between the server and clients occurs over a persistent TCP connection as long as the client remains connected and the server is online.

Client Connection Establishment

A client initiates communication by opening a TCP connection to the server's designated port. Once connection is established the client may begin issuing commands. If the connection fails, the client reports a connection failure locally.

Graceful Client Disconnection

When a client disconnects from the server using the QUIT command the server notifies the members of the rooms the client was a member of and removes the client from those rooms, then closes the TCP connection with the client.

Server-Initiated Disconnection

If the server is shutdown it sends a notice to all currently connected clients.

Unexpected Client Crashes

If the client terminates unexpectedly, the server detects the closed TCP socket, removes the client from all rooms, and sends a notice to the remaining members of each of those rooms.

Unexpected Server Crashes

If the server crashes or the connection is lost without a shutdown notice, the client detects the broken TCP connection and terminates the session.

Concurrency

The IRC system is designed to support multiple clients simultaneously. Because all communication is processed at the server, the server must be capable of receiving, parsing, and processing commands from multiple clients concurrently.

Messages within a single room are delivered in the order the server processes them. TCP ensures that messages from a single client arrive in the order sent.